МИНОБРНАУКИ РОССИИ
Федеральное государственное бюджетное образовательное учреждение
высшего образования
«МИРЭА – Российский технологический университет»
РТУ МИРЭА

Колледж программирования и кибербезопасности

ОТЧЁТ

по практическому занятию № 3
«Динамическое выделение памяти»

по междисциплинарному курсу МДК.01.04 «Системное программирование»
ПЦК «Программирование»

Выполнил:
студент группы ПКС-33
Шевырин Александр

Проверил:
преподаватель
Мухортова Наталья Николаевна

Москва, 2026

СОДЕРЖАНИЕ

ЦЕЛЬ И ЗАДАНИЕ 3
ТЕОРЕТИЧЕСКИЕ СВЕДЕНИЯ 4
ХОД ВЫПОЛНЕНИЯ РАБОТЫ 5
ТЕСТИРОВАНИЕ 17
ЗАКЛЮЧЕНИЕ 18
СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ 19

ЦЕЛЬ И ЗАДАНИЕ

Цель работы: освоить принципы построения структур с динамическим выделением памяти.

Задание. Разработать, отладить и продемонстрировать работу двух программ.

1. В первой программе описать динамическую структуру, которая содержит указатель на статическую структуру Book (с тремя полями на выбор студента). Создать две динамические структуры, заполнить их информацией о книгах, связать между собой и вывести информацию из обеих структур только через первую из них.

2. Создать динамическую структуру согласно списку задач. Номер индивидуальной задачи вычисляется как остаток от деления на 5 номера студента по списку в журнале группы плюс один.

Индивидуальные задачи:

1) стек на односвязном списке: создание, добавление элемента, удаление элемента из середины списка (не первого и не последнего), вывод списка;

2) двусвязный список: создание, добавление элемента, удаление элемента из конца списка, вывод списка;

3) двусвязный список: создание, добавление элемента, удаление элемента из начала списка, вывод списка из начала в конец и из конца в начало;

4) циклический двусвязный список: создание, добавление элемента, удаление элемента из середины списка, вывод списка;

5) циклический односвязный список: создание, добавление элемента, удаление элемента из середины списка, вывод списка.

Так как порядковый номер в журнале группы в задании не указан, в работе выполнены все пять вариантов индивидуальной задачи; в отчёте они обозначены как задачи 2.1–2.5.

ТЕОРЕТИЧЕСКИЕ СВЕДЕНИЯ

Динамическая структура данных – это структура, размер и состав которой изменяются во время выполнения программы. Память под её элементы выделяется в «куче» (heap) функциями **malloc()**, **calloc()**, **realloc()** из заголовочного файла stdlib.h, а освобождается функцией **free()**. Каждый выделенный блок необходимо освободить ровно один раз, иначе возникает утечка памяти.

Элемент динамической структуры (узел) содержит информационную часть и один или несколько указателей на соседние элементы:

– **односвязный список**: узел хранит указатель на следующий элемент; последний элемент указывает на NULL;

– **двусвязный список**: узел хранит указатели на предыдущий и следующий элементы, что позволяет обходить список в обоих направлениях;

– **циклический список**: последний элемент ссылается на первый (в двусвязном – ещё и первый на последний), признака конца NULL нет;

– **стек** – структура, работающая по принципу LIFO («последним пришёл – первым вышел»); на односвязном списке вершина стека является головой списка.

Для удаления элемента из середины списка необходимо найти предшествующий ему узел, «перебросить» указатель на следующий за удаляемым узел и только после этого освободить память удаляемого узла.

ХОД ВЫПОЛНЕНИЯ РАБОТЫ

Программа 1. Динамическая структура со ссылкой на структуру Book

Описана статическая структура **Book** с тремя полями: название (title), автор (author), год издания (year). Динамическая структура **Node** содержит указатель *book* на структуру Book и указатель *next* на следующий узел. Функция create_node() выделяет память под узел и под книгу и заполняет поля. В main() создаются два узла, связываются через поле next, и информация о обеих книгах выводится в цикле, начинающемся только с первого узла. В конце память освобождается.

Исходный код программы приведён в листинге 1.

Листинг 1 – prog1_book.c

```
#include <stdio.h>
#include <stdlib.h>
#include <string.h>

/* Статическая структура Book (три поля) */
typedef struct {
    char title[50];
    char author[50];
    int year;
} Book;

/* Динамическая структура: содержит указатель на Book и на следующий узел */
typedef struct Node {
    Book *book;
    struct Node *next;
} Node;

Node *create_node(const char *title, const char *author, int year)
{
    Node *n = (Node *)malloc(sizeof(Node));
    if (n == NULL) return NULL;
    n->book = (Book *)malloc(sizeof(Book));
    if (n->book == NULL) { free(n); return NULL; }
    strncpy(n->book->title, title, sizeof(n->book->title) - 1);
    n->book->title[sizeof(n->book->title) - 1] = '\0';
    strncpy(n->book->author, author, sizeof(n->book->author) - 1);
    n->book->author[sizeof(n->book->author) - 1] = '\0';
    n->book->year = year;
    n->next = NULL;
    return n;
}

int main(void)
{
    Node *first = create_node("Война и мир", "Л. Н. Толстой", 1869);
    Node *second = create_node("Мастер и Маргарита", "М. А. Булгаков", 1967);
    if (first == NULL || second == NULL) {
        printf("Ошибка выделения памяти\n");
```

```
        return 1;
    }
    first->next = second;           /* связываем структуры между собой */

    /* вывод информации из обеих структур только через первую */
    int i = 1;
    for (Node *p = first; p != NULL; p = p->next, i++)
        printf("Книга %d: \"%s\", %s, %d г.\n", i, p->book->title,
               p->book->author, p->book->year);

    /* освобождение памяти */
    free(first->next->book);
    free(first->next);
    free(first->book);
    free(first);
    return 0;
}
```

Результат работы программы показан на рисунке 1.

```
Книга 1: "Война и мир", Л. Н. Толстой, 1869 г.
Книга 2: "Мастер и Маргарита", М. А. Булгаков, 1967 г.
```

Рисунок 1 – Вывод программы «Программа 1. Динамическая структура со ссылкой на структуру Book»

Задача 2.1. Стек на односвязном списке

Стек реализован как односвязный список, вершина (top) – голова списка. Функции: create() – создание пустого стека; push() – добавление элемента в вершину; remove_middle() – удаление элемента с номером 2…size–1 (считая от вершины), для первого и последнего элементов возвращается отказ; print() – вывод; destroy() – освобождение памяти.

Исходный код программы приведён в листинге 2.

Листинг 2 – prog2_1_stack.c

```
#include <stdio.h>
#include <stdlib.h>

/* Задача 1. Стек на односвязном списке */
typedef struct Node {
    int data;
    struct Node *next;
} Node;

typedef struct {
    Node *top;      /* вершина стека */
    int size;
} Stack;

void create(Stack *s) { s->top = NULL; s->size = 0; }

int push(Stack *s, int value)
{
    Node *n = (Node *)malloc(sizeof(Node));
    if (n == NULL) return 0;
    n->data = value;
    n->next = s->top;
    s->top = n;
    s->size++;
    return 1;
}

/* Удаление элемента из середины (не первого и не последнего).
   pos - номер элемента от вершины, 2 .. size-1 */
int remove_middle(Stack *s, int pos)
{
    if (pos < 2 || pos > s->size - 1) return 0;
    Node *prev = s->top;
    for (int i = 1; i < pos - 1; i++) prev = prev->next;
    Node *del = prev->next;
    prev->next = del->next;
    free(del);
    s->size--;
    return 1;
}

void print(const Stack *s)
{
    if (s->top == NULL) { printf("Стек пуст\n"); return; }
    printf("Вершина -> ");
```

```
    for (Node *p = s->top; p != NULL; p = p->next)
        printf("%d%s", p->data, p->next ? " -> " : "\n");
}

void destroy(Stack *s)
{
    while (s->top) { Node *t = s->top; s->top = t->next; free(t); }
    s->size = 0;
}

int main(void)
{
    Stack s;
    create(&s);
    for (int i = 1; i <= 6; i++) push(&s, i * 10);
    printf("Исходный стек:\n");
    print(&s);
    int pos = 3;
    if (remove_middle(&s, pos))
        printf("Удалён элемент №%d (от вершины):\n", pos);
    print(&s);
    printf("Попытка удалить первый элемент: %s\n",
           remove_middle(&s, 1) ? "удалён" : "отказ (не середина)");
    printf("Попытка удалить последний элемент: %s\n",
           remove_middle(&s, s.size) ? "удалён" : "отказ (не середина)");
    destroy(&s);
    return 0;
}
```

Результат работы программы показан на рисунке 2.

```
Исходный стек:
Вершина -> 60 -> 50 -> 40 -> 30 -> 20 -> 10
Удалён элемент №3 (от вершины):
Вершина -> 60 -> 50 -> 30 -> 20 -> 10
Попытка удалить первый элемент: отказ (не середина)
Попытка удалить последний элемент: отказ (не середина)
```

Рисунок 2 – Вывод программы «Задача 2.1. Стек на односвязном списке»

Задача 2.2. Двусвязный список, удаление из конца

Список хранит указатели на head и tail. Функции: create(), add() – добавление в конец, remove_last() – удаление последнего элемента (с корректировкой tail), remove_middle() – удаление элемента из середины, print(), destroy().

Исходный код программы приведён в листинге 3.

Листинг 3 – prog2_2_dlist_end.c

```
#include <stdio.h>
#include <stdlib.h>

/* Задача 2. Двусвязный список, удаление из конца */
typedef struct Node {
    int data;
    struct Node *prev, *next;
} Node;

typedef struct {
    Node *head, *tail;
    int size;
} List;

void create(List *l) { l->head = l->tail = NULL; l->size = 0; }

int add(List *l, int value)         /* добавление в конец */
{
    Node *n = (Node *)malloc(sizeof(Node));
    if (n == NULL) return 0;
    n->data = value;
    n->next = NULL;
    n->prev = l->tail;
    if (l->tail) l->tail->next = n; else l->head = n;
    l->tail = n;
    l->size++;
    return 1;
}

int remove_last(List *l)            /* удаление из конца */
{
    if (l->tail == NULL) return 0;
    Node *del = l->tail;
    l->tail = del->prev;
    if (l->tail) l->tail->next = NULL; else l->head = NULL;
    free(del);
    l->size--;
    return 1;
}

int remove_middle(List *l, int pos) /* удаление из середины, pos = 2..size-1 */
{
    if (pos < 2 || pos > l->size - 1) return 0;
    Node *p = l->head;
    for (int i = 1; i < pos; i++) p = p->next;
    p->prev->next = p->next;
```

```
    p->next->prev = p->prev;
    free(p);
    l->size--;
    return 1;
}

void print(const List *l)
{
    if (l->head == NULL) { printf("Список пуст\n"); return; }
    for (Node *p = l->head; p; p = p->next)
        printf("%d%s", p->data, p->next ? " <-> " : "\n");
}

void destroy(List *l) { while (remove_last(l)); }

int main(void)
{
    List l;
    create(&l);
    for (int i = 1; i <= 5; i++) add(&l, i * 10);
    printf("Исходный список:\n");
    print(&l);
    remove_last(&l);
    printf("После удаления из конца:\n");
    print(&l);
    remove_middle(&l, 2);
    printf("После удаления из середины (2-й элемент):\n");
    print(&l);
    destroy(&l);
    printf("После очистки:\n");
    print(&l);
    return 0;
}
```

Результат работы программы показан на рисунке 3.

```
Исходный список:
10 <-> 20 <-> 30 <-> 40 <-> 50
После удаления из конца:
10 <-> 20 <-> 30 <-> 40
После удаления из середины (2-й элемент):
10 <-> 30 <-> 40
После очистки:
Список пуст
```

Рисунок 3 – Вывод программы «Задача 2.2. Двусвязный список, удаление из конца»

Задача 2.3. Двусвязный список, удаление из начала

Реализованы функции create(), add(), remove_first() – удаление первого элемента (с корректировкой head), remove_middle(), а также два вида вывода: print_forward() – от начала к концу и print_backward() – от конца к началу по указателям prev.

Исходный код программы приведён в листинге 4.

Листинг 4 – prog2_3_dlist_begin.c

```
#include <stdio.h>
#include <stdlib.h>

/* Задача 3. Двусвязный список, удаление из начала, вывод в обе стороны */
typedef struct Node {
    int data;
    struct Node *prev, *next;
} Node;

typedef struct {
    Node *head, *tail;
    int size;
} List;

void create(List *l) { l->head = l->tail = NULL; l->size = 0; }

int add(List *l, int value)         /* добавление в конец */
{
    Node *n = (Node *)malloc(sizeof(Node));
    if (n == NULL) return 0;
    n->data = value;
    n->next = NULL;
    n->prev = l->tail;
    if (l->tail) l->tail->next = n; else l->head = n;
    l->tail = n;
    l->size++;
    return 1;
}

int remove_first(List *l)           /* удаление из начала */
{
    if (l->head == NULL) return 0;
    Node *del = l->head;
    l->head = del->next;
    if (l->head) l->head->prev = NULL; else l->tail = NULL;
    free(del);
    l->size--;
    return 1;
}

int remove_middle(List *l, int pos) /* удаление из середины, pos = 2..size-1 */
{
    if (pos < 2 || pos > l->size - 1) return 0;
    Node *p = l->head;
    for (int i = 1; i < pos; i++) p = p->next;
    p->prev->next = p->next;
```

```
    p->next->prev = p->prev;
    free(p);
    l->size--;
    return 1;
}

void print_forward(const List *l)
{
    if (l->head == NULL) { printf("Список пуст\n"); return; }
    for (Node *p = l->head; p; p = p->next)
        printf("%d%s", p->data, p->next ? " <-> " : "\n");
}

void print_backward(const List *l)
{
    if (l->tail == NULL) { printf("Список пуст\n"); return; }
    for (Node *p = l->tail; p; p = p->prev)
        printf("%d%s", p->data, p->prev ? " <-> " : "\n");
}

void destroy(List *l) { while (remove_first(l)); }

int main(void)
{
    List l;
    create(&l);
    for (int i = 1; i <= 5; i++) add(&l, i * 10);
    printf("Исходный список (из начала в конец):\n");
    print_forward(&l);
    printf("Исходный список (из конца в начало):\n");
    print_backward(&l);
    remove_first(&l);
    printf("После удаления из начала:\n");
    print_forward(&l);
    remove_middle(&l, 2);
    printf("После удаления из середины (2-й элемент):\n");
    print_forward(&l);
    printf("Из конца в начало:\n");
    print_backward(&l);
    destroy(&l);
    return 0;
}
```

Результат работы программы показан на рисунке 4.

```
Исходный список (из начала в конец):
10 <-> 20 <-> 30 <-> 40 <-> 50
Исходный список (из конца в начало):
50 <-> 40 <-> 30 <-> 20 <-> 10
После удаления из начала:
20 <-> 30 <-> 40 <-> 50
После удаления из середины (2-й элемент):
20 <-> 40 <-> 50
Из конца в начало:
50 <-> 40 <-> 20
```

Рисунок 4 – Вывод программы «Задача 2.3. Двусвязный список, удаление из начала»

Задача 2.4. Циклический двусвязный список

В списке хранится указатель на голову head; указатель head->prev является хвостом списка, а tail->next возвращает к голове. Функции: create(), add(), remove_middle(), print() (обход до возврата к голове), destroy().

Исходный код программы приведён в листинге 5.

Листинг 5 – prog2_4_cyc_dlist.c

```
#include <stdio.h>
#include <stdlib.h>

/* Задача 4. Циклический двусвязный список */
typedef struct Node {
    int data;
    struct Node *prev, *next;
} Node;

typedef struct {
    Node *head;     /* head->prev == tail */
    int size;
} List;

void create(List *l) { l->head = NULL; l->size = 0; }

int add(List *l, int value)         /* добавление в конец */
{
    Node *n = (Node *)malloc(sizeof(Node));
    if (n == NULL) return 0;
    n->data = value;
    if (l->head == NULL) {
        n->next = n->prev = n;
        l->head = n;
    } else {
        Node *tail = l->head->prev;
        n->next = l->head;
        n->prev = tail;
        tail->next = n;
        l->head->prev = n;
    }
    l->size++;
    return 1;
}

int remove_middle(List *l, int pos) /* pos = 2..size-1 */
{
    if (pos < 2 || pos > l->size - 1) return 0;
    Node *p = l->head;
    for (int i = 1; i < pos; i++) p = p->next;
    p->prev->next = p->next;
    p->next->prev = p->prev;
    free(p);
    l->size--;
    return 1;
}

void print(const List *l)
```

```
{
    if (l->head == NULL) { printf("Список пуст\n"); return; }
    Node *p = l->head;
    do {
        printf("%d <-> ", p->data);
        p = p->next;
    } while (p != l->head);
    printf("(к началу: %d)\n", l->head->data);
}

void destroy(List *l)
{
    for (int i = 0; i < l->size; i++) {
        Node *t = l->head;
        l->head = t->next;
        free(t);
    }
    l->head = NULL;
    l->size = 0;
}

int main(void)
{
    List l;
    create(&l);
    for (int i = 1; i <= 6; i++) add(&l, i * 10);
    printf("Исходный список:\n");
    print(&l);
    remove_middle(&l, 3);
    printf("После удаления 3-го элемента:\n");
    print(&l);
    printf("Попытка удалить первый: %s\n", remove_middle(&l, 1) ? "удалён" : "отказ")
    printf("Попытка удалить последний: %s\n", remove_middle(&l, l.size) ? "удалён" :
    destroy(&l);
    return 0;
}
```

Результат работы программы показан на рисунке 5.

```
Исходный список:
10 <-> 20 <-> 30 <-> 40 <-> 50 <-> 60 <-> (к началу: 10)
После удаления 3-го элемента:
10 <-> 20 <-> 40 <-> 50 <-> 60 <-> (к началу: 10)
Попытка удалить первый: отказ
Попытка удалить последний: отказ
```

Рисунок 5 – Вывод программы «Задача 2.4. Циклический двусвязный список»

Задача 2.5. Циклический односвязный список

В списке хранится указатель на хвост tail; tail->next является головой, что делает добавление в конец и обход за O(1) и O(n) соответственно. Функции: create(), add(), remove_middle(), print(), destroy().

Исходный код программы приведён в листинге 6.

Листинг 6 – prog2_5_cyc_slist.c

```
#include <stdio.h>
#include <stdlib.h>

/* Задача 5. Циклический односвязный список */
typedef struct Node {
    int data;
    struct Node *next;
} Node;

typedef struct {
    Node *tail;     /* tail->next == head */
    int size;
} List;

void create(List *l) { l->tail = NULL; l->size = 0; }

int add(List *l, int value)         /* добавление в конец */
{
    Node *n = (Node *)malloc(sizeof(Node));
    if (n == NULL) return 0;
    n->data = value;
    if (l->tail == NULL) {
        n->next = n;
    } else {
        n->next = l->tail->next;
        l->tail->next = n;
    }
    l->tail = n;
    l->size++;
    return 1;
}

int remove_middle(List *l, int pos) /* pos = 2..size-1 */
{
    if (pos < 2 || pos > l->size - 1) return 0;
    Node *prev = l->tail->next;     /* голова */
    for (int i = 1; i < pos - 1; i++) prev = prev->next;
    Node *del = prev->next;
    prev->next = del->next;
    free(del);
    l->size--;
    return 1;
}

void print(const List *l)
{
    if (l->tail == NULL) { printf("Список пуст\n"); return; }
    Node *head = l->tail->next, *p = head;
```

```
    do {
        printf("%d -> ", p->data);
        p = p->next;
    } while (p != head);
    printf("(к началу: %d)\n", head->data);
}

void destroy(List *l)
{
    for (int i = 0; i < l->size; i++) {
        Node *t = l->tail->next;
        l->tail->next = t->next;
        free(t);
    }
    l->tail = NULL;
    l->size = 0;
}

int main(void)
{
    List l;
    create(&l);
    for (int i = 1; i <= 6; i++) add(&l, i * 10);
    printf("Исходный список:\n");
    print(&l);
    remove_middle(&l, 4);
    printf("После удаления 4-го элемента:\n");
    print(&l);
    printf("Попытка удалить первый: %s\n", remove_middle(&l, 1) ? "удалён" : "отказ")
    printf("Попытка удалить последний: %s\n", remove_middle(&l, l.size) ? "удалён" :
    destroy(&l);
    return 0;
}
```

Результат работы программы показан на рисунке 6.

```
Исходный список:
10 -> 20 -> 30 -> 40 -> 50 -> 60 -> (к началу: 10)
После удаления 4-го элемента:
10 -> 20 -> 30 -> 50 -> 60 -> (к началу: 10)
Попытка удалить первый: отказ
Попытка удалить последний: отказ
```

Рисунок 6 – Вывод программы «Задача 2.5. Циклический односвязный список»

ТЕСТИРОВАНИЕ

Все программы скомпилированы компилятором GCC с ключами `-Wall -Wextra -g` без предупреждений. Для проверки корректности работы с динамической памятью программы были запущены с санитайзерами `-fsanitize=address,undefined`: ошибок обращения к памяти, неопределённого поведения и утечек памяти не обнаружено (код возврата каждой программы – 0).

Граничные случаи: удаление первого и последнего элементов в задачах 2.1, 2.4 и 2.5 отклоняется функцией remove_middle() (согласно условию, удаляется только «середина»); вывод пустого списка сообщает «Список пуст»; в задачах 2.2 и 2.3 после удаления последнего элемента указатели head и tail становятся равными NULL.

ЗАКЛЮЧЕНИЕ

В ходе практического занятия освоены принципы построения структур с динамическим выделением памяти. Разработаны и отлажены программа со связанными динамическими структурами, содержащими указатели на статическую структуру Book, и программы для пяти видов списков: стека на односвязном списке, двусвязного списка с удалением из конца и из начала, циклического двусвязного и циклического односвязного списков. Для каждой структуры реализованы функции создания, добавления, удаления элементов, вывода и освобождения памяти. Утечек памяти и ошибок доступа при тестировании не выявлено.

СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ

1. Керниган Б., Ритчи Д. Язык программирования Си. – 2-е изд. – М.: Вильямс, 2015. – 304 с.

2. Прата С. Язык программирования С. Лекции и упражнения. – 6-е изд. – М.: Вильямс, 2015. – 928 с.

3. Кормен Т., Лейзерсон Ч., Ривест Р., Штайн К. Алгоритмы: построение и анализ. – 3-е изд. – М.: Вильямс, 2013. – 1328 с.

4. Мухортова Н. Н. Язык Си и динамическое выделение памяти: лекционные материалы по МДК.01.04 «Системное программирование». – М.: РТУ МИРЭА, 2026.

5. ISO/IEC 9899:2018. Information technology – Programming languages – C. – Geneva: ISO, 2018.